1. ZADATAK

PRII – G1

Izvršiti definiciju funkcija na način koji odgovara opisu (komentarima) datom neposredno uz pozive ili nazive funkcija. Možete dati komentar na bilo koju liniju code-a koju smatrate da bi trebalo unaprijediti ili da će eventualno uzrokovati grešku prilikom kompajliranja. Također, možete dodati dodatne funkcije ili metode koje će vam olakšati implementaciju programa.

```
#include <iostream>
using namespace std;

string crt = "\n-------------------------------------------------------
---------------------------\n";

string PORUKA_TELEFON = crt +
"TELEFONE ISKLJUCITE I ODLOZITE U TORBU, DZEP ILI DRUGU LOKACIJU VAN
DOHVATA.\n"
"CESTO SE NA TELEFONIMA (PRO)NALAZE PROGRAMSKI KODOVI KOJI MOGU BITI
ISKORISTENI ZA\n"
"RJESAVANJE ISPITNOG ZADATKA, STO CE, U SLUCAJU PRONALASKA, BITI
SANKCIONISANO.\n\n"
"SVI KOD KOJIH BUDE PRONADJEN TELEFON ILI NEKI DRUGI UREDJAJ KOJI MOZE
SADRZAVATI\n"
"MATERIJAL ILI RJESENJA ISPITNIH ZADATAKA CE BITI UDALJENI SA ISPITA I
BIT CE POKRENUT\n"
"POSTUPAK PROTIV NJIH" + crt;

string PORUKA_ISPIT = crt +
"0. PROVJERITE DA LI ZADACI PRIPADAJU VASOJ GRUPI (G1/G2)\n"
"1.  SVE  KLASE  SA  DINAMICKOM  ALOKACIJOM  MORAJU  IMATI  ISPRAVAN
DESTRUKTOR\n"
"2.  IZOSTAVLJANJE  DESTRUKTORA  ILI  NJEGOVIH  DIJELOVA  BIT  CE  OZNACENO
KAO TM\n"
"3. ATRIBUTI, METODE I PARAMETRI MORAJU BITI IDENTICNI ONIMA U TESTNOJ
MAIN FUNKCIJI,\n"
"   OSIM AKO POSTOJI JASNO OPISAN RAZLOG ZA MODIFIKACIJU\n"
"4. IZUZETKE BACAJTE SAMO TAMO GDJE JE IZRICITO NAGLASENO\n"
"5. SVE METODE KOJE SE POZIVAJU U MAIN-U MORAJU POSTOJATI.\n"
"    AKO  NEMATE  ZELJENU  IMPLEMENTACIJU,  OSTAVITE  PRAZNO  TIJELO  ILI
VRATITE DEFAULT VRIJEDNOST\n"
"6.  U  MAIN  FUNKCIJI  MOZETE  DODAVATI  TESTNE  PODATKE  I  POZIVE  PO
VLASTITOM IZBORU\n"
"7. TESTIRAJTE PROGRAM U OBA MODA (F5 i Ctrl+F5)" + crt;

char* AlocirajTekst(const char* tekst) {
    if (tekst == nullptr) return nullptr;
    size_t velicina = strlen(tekst) + 1;
    char* noviTekst = new char[velicina];
    strcpy_s(noviTekst, velicina, tekst);
    return noviTekst;
}

enum Kategorija { EKONOMSKA, SREDNJA, LUKSUZNA, KOMBI };
const char* KategorijaOznake[] = { "EK", "SR", "LX", "KB" };
const char* KategorijaNazivi[] = { "EKONOMSKA", "SREDNJA", "LUKSUZNA",
"KOMBI" };

template<class T1, class T2, int max>
```

```
class Kolekcija {
    T1* _elementi1;
    T2* _elementi2;
    int _trenutno;
    void Dealociraj() {
        delete[] _elementi1;
        delete[] _elementi2;
        _elementi1 = nullptr;
        _elementi2 = nullptr;
    }
public:
    int GetTrenutno() const { return _trenutno; }
    const    T1&    GetElement1(int    indeks)    const    {    return
_elementi1[indeks]; }
    const    T2&    GetElement2(int    indeks)    const    {    return
_elementi2[indeks]; }

    friend ostream& operator<<(ostream& COUT, const Kolekcija& obj) {
        for (int i = 0; i < obj.GetTrenutno(); i++)
            COUT << obj.GetElement1(i) << " " << obj.GetElement2(i) <<
endl;
        return COUT;
    }
    ~Kolekcija() { Dealociraj(); }
};

class DatumVrijeme {
    int* _dan, * _mjesec, * _godina, * _sati, * _minute;
    void Dealociraj() {
        delete _dan;
        delete _mjesec;
        delete _godina;
        delete _sati;
        delete _minute;
        _dan = _mjesec = _godina = _sati = _minute = nullptr;
    }
public:
    DatumVrijeme(int dan = 1, int mjesec = 1, int godina = 2000,
        int sati = 0, int minute = 0) {
        _dan = new int(dan);
        _mjesec = new int(mjesec);
        _godina = new int(godina);
        _sati = new int(sati);
        _minute = new int(minute);
    }
    int GetGodina() const { return *_godina; }
    friend ostream& operator<<(ostream& COUT, const DatumVrijeme& obj)
{
        // ToString vraca datum i vrijeme u formatu DD.MM.GGGG HH:MM
        COUT << obj.ToString();
        return COUT;
    }
    ~DatumVrijeme() {
        Dealociraj();
    }
};

class Klijent {
```

```
    string _imePrezime;
    string _vozackaDozvola;
    string _email;
    int _godinaRodjenja;
public:
    const string& GetImePrezime() const { return _imePrezime; }
    const string& GetVozackaDozvola() const { return _vozackaDozvola;
}
    const string& GetEmail() const { return _email; }
    int GetGodinaRodjenja() const { return _godinaRodjenja; }
    friend ostream& operator<<(ostream& COUT, const Klijent& obj) {
        COUT << obj._imePrezime << " | " << obj._vozackaDozvola << " |
"
            << obj._email;
        return COUT;
    }
};

class Vozilo {
    char* _registracija;
    char* _model;
    Kategorija _kategorija;
    double _dnevnaTarifa;
    void Dealociraj() {
        delete[] _registracija;
        delete[] _model;
        _registracija = _model = nullptr;
    }
public:
    const char* GetRegistracija() const { return _registracija; }
    const char* GetModel() const { return _model; }
    Kategorija GetKategorija() const { return _kategorija; }
    double GetDnevnaTarifa() const { return _dnevnaTarifa; }
    friend ostream& operator<<(ostream& COUT, const Vozilo& obj) {
        COUT << obj._registracija << " | " << obj._model << " | "
            << KategorijaNazivi[(int)obj._kategorija] << " | "
            << obj._dnevnaTarifa << " KM/dan";
        return COUT;
    }
    ~Vozilo() { Dealociraj(); }
};

class Najam {
    char* _oznaka;
    int _redniBroj;
    Klijent _klijent;
    Vozilo _vozilo;
    DatumVrijeme _preuzimanje;
    DatumVrijeme _vracanje;
    Kolekcija<string, double, 10> _dodatneStavke;
    void Dealociraj() {
        delete[] _oznaka;
        _oznaka = nullptr;
    }
public:
    const char* GetOznaka() const { return _oznaka; }
    int GetRedniBroj() const { return _redniBroj; }
    Klijent& GetKlijent() { return _klijent; }
```

```
    const Klijent& GetKlijent() const { return _klijent; }
    Vozilo& GetVozilo() { return _vozilo; }
    const Vozilo& GetVozilo() const { return _vozilo; }
    const DatumVrijeme& GetPreuzimanje() const { return _preuzimanje;
}
    const DatumVrijeme& GetVracanje() const { return _vracanje; }
    Kolekcija<string, double, 10>& GetDodatneStavke() { return
_dodatneStavke; }
    const Kolekcija<string, double, 10>& GetDodatneStavke() const {
        return _dodatneStavke;
    }
    friend ostream& operator<<(ostream& COUT, const Najam& obj) {
        // ToString vraca:
        // oznaka | ime i prezime klijenta | model vozila |
preuzimanje - vracanje
        COUT << obj.ToString();
        return COUT;
    }
    ~Najam() { Dealociraj(); }
};

class Agencija {
    char* _naziv;
    vector<Najam> _najmovi;
    void Dealociraj() {
        delete[] _naziv;
        _naziv = nullptr;
    }
public:
    Agencija(const char* naziv = "") {
        _naziv = AlocirajTekst(naziv);
    }
    const char* GetNaziv() const { return _naziv; }
    vector<Najam>& GetNajmovi() { return _najmovi; }
    const vector<Najam>& GetNajmovi() const { return _najmovi; }
    ~Agencija() { Dealociraj(); }
};

const char* GetOdgovorNaPrvoPitanje() {
    cout << "Pitanje -> Pojasnite pojmove enkapsulacije i
polimorfizma. Na koji nacin se implementiraju i koji su najznacajniji
efekti njihove primjene?\n";
    return "Odgovor -> OVDJE UNESITE VAS ODGOVOR";
}

const char* GetOdgovorNaDrugoPitanje() {
    cout << "Pitanje -> Pojasnite ulogu i znacaj move konstruktora. Po
cemu se razlikuje od ostalih clanica, kako se poziva i koji su
najznacajniji efekti njegove primjene?\n";
    return "Odgovor -> OVDJE UNESITE VAS ODGOVOR";
}

int main() {
    cout << PORUKA_TELEFON; cin.get();
    cout << PORUKA_TELEFON; cin.get();
    cout << PORUKA_ISPIT; cin.get();
    system("cls");
```

```
    cout << GetOdgovorNaPrvoPitanje() << crt;
    cin.get();
    cout << GetOdgovorNaDrugoPitanje() << crt;
    cin.get();

    /*
    Funkcija GenerisiOznaku generise oznaku najma u formatu:
    RC-KAT-BBBB-IN

    Potpis funkcije treba biti:
    string GenerisiOznaku(const char* imeKlijenta, int redniBroj,
Kategorija kategorija)

    RC   -> fiksni prefiks,
    KAT  -> oznaka kategorije vozila (EK, SR, LX, KB),
    BBBB -> redni broj najma popunjen nulama (0001-9999),
    IN   -> inicijali imena i prezimena klijenta.

    Kod imena sa vise rijeci koriste se inicijal prve i posljednje
rijeci.
    Za neispravne podatke funkcija vraca "RC-XX-0000-XX".
    */
    if (GenerisiOznaku("Amina Buric", 42, EKONOMSKA) == "RC-EK-0042-
AB")
        cout << "Oznaka OK" << crt;
    if (GenerisiOznaku("Goran Skondric", 7, LUKSUZNA) == "RC-LX-0007-
GS")
        cout << "Oznaka OK" << crt;
    if (GenerisiOznaku("Ana Marija Kovac", 156, KOMBI) == "RC-KB-0156-
AK")
        cout << "Oznaka OK" << crt;
    if (GenerisiOznaku("Amina", 42, EKONOMSKA) == "RC-XX-0000-XX" &&
        GenerisiOznaku("Amina Buric", 0, EKONOMSKA) == "RC-XX-0000-XX"
&&
        GenerisiOznaku("Amina  Buric",  10000,  EKONOMSKA)  ==  "RC-XX-
0000-XX" &&
        GenerisiOznaku("Amina Buric", 42, (Kategorija)99) == "RC-XX-
0000-XX")
        cout << "Neispravni podaci za oznaku OK" << crt;

    /*
    ValidirajOznaku koristi regex za provjeru formata. Kategorija mora
biti
    EK, SR, LX ili KB, redni broj cetiri cifre (neispravno je i 0000),
inicijali velika
    slova.
    */
    if (ValidirajOznaku("RC-EK-0042-AB"))
        cout << "OZNAKA VALIDNA" << crt;
    if (!ValidirajOznaku("RC-EK-0042-Ab") &&
        !ValidirajOznaku("rc-EK-0042-AB") &&
        !ValidirajOznaku("RC-XX-0042-AB") &&
        !ValidirajOznaku("RC-EK-42-AB") &&
        !ValidirajOznaku("RC-EK-0000-AB"))
        cout << "OZNAKA NIJE VALIDNA" << crt;

    /*
    RazlikaUDanima vraca razliku u danima izmedju dva datuma.
```

```
    */
    DatumVrijeme d1(1, 6, 2026);
    DatumVrijeme d2(6, 6, 2026);
    if (d2.RazlikaUDanima(d1) == 5)
        cout << "RazlikaUDanima OK (" << d2.RazlikaUDanima(d1) << ")"
<< crt;

    /*
    Metoda Dodaj dodaje novi par, te baca izuzetak u slucaju da je
kolekcija popunjena.
    */
    Kolekcija<string, double, 5> troskovi;
    troskovi.Dodaj("GPS", 10);
    troskovi.Dodaj("Djecije sjediste", 5);
    troskovi.Dodaj("Dodatni vozac", 20);
    troskovi.Dodaj("Zimske gume", 15);
    cout << troskovi << crt;

    Kolekcija<string, double, 5> kopijaTroskova = troskovi;
    kopijaTroskova.GetElement2(0) = 99;
    Kolekcija<string, double, 5> dodijeljeni;
    dodijeljeni = troskovi;
    cout << "Original:" << crt << troskovi;
    cout << "Kopija (izmijenjena):" << crt << kopijaTroskova;
    cout << "Dodijeljeni:" << crt << dodijeljeni;
    try {
        Kolekcija<int, int, 1> mala;
        mala.Dodaj(1, 1);
        mala.Dodaj(2, 2);
    }
    catch (exception& e) {
        cout << "Exception: " << e.what() << crt;
    }

    /*
    IzdvojiOpseg uklanja parove iz kolekcije u opsegu OD - DO (ne
ukljucujuci lokaciju DO) i vraca
    ih kao novu kolekciju; preostali elementi popunjavaju prazninu.
Neispravan
    opseg baca izuzetak.
    */
    Kolekcija<string,       double,       5>       izdvojeni       =
kopijaTroskova.IzdvojiOpseg(1, 3);
    cout << "Izdvojeno:" << crt << izdvojeni;
    cout << "Ostatak nakon izdvajanja:" << crt << kopijaTroskova;
    try {
        kopijaTroskova.IzdvojiOpseg(0, 10);
    }
    catch (exception& e) {
        cout << "Exception: " << e.what() << crt;
    }

    /*
    UkloniPoKljucu uklanja pronadjeni prvi par sa identicnom
vrijednoscu prvog elementa
    (koji je proslijedjen) uz pomjeranje ostalih. u zavisnosti od
uspjesnosti funkcija
    vraca true ili false.
```

```
    */
    if (dodijeljeni.UkloniPoKljucu("Djecije sjediste"))
        cout << "Stavka uklonjena, ostalo: " <<
dodijeljeni.GetTrenutno() << crt;
    if (!dodijeljeni.UkloniPoKljucu("Nepostojeca"))
        cout << "Uklanjanje nepostojece stavke vraca false" << crt;
    cout << "Nakon uklanjanja:" << crt << dodijeljeni;

    Klijent amina("Amina Buric", "VD-123456", "amina@mail.ba", 2005);
    Klijent goran("Goran Skondric", "VD-654321", "goran@mail.ba",
1985);
    Klijent kopijaKlijenta = amina;
    cout << kopijaKlijenta << crt;

    /*
    Vozilo cuva registraciju, model, kategoriju i dnevnu tarifu.
    */
    Vozilo golf("A12-B-345", "VW Golf", EKONOMSKA, 60);
    Vozilo passat("K99-J-111", "VW Passat", SREDNJA, 90);
    Vozilo kopijaVozila = golf;
    cout << kopijaVozila << crt;

    /*
    Konstruktor Najma generise oznaku na osnovu klijenta, rednog broja
i
    kategorije vozila. Period najma je odredjen datumima preuzimanja i
    vracanja.
    */
    DatumVrijeme preuzimanje(1, 6, 2026, 8, 0);
    DatumVrijeme vracanje(6, 6, 2026, 8, 0);
    Najam najam1(amina, golf, 42, preuzimanje, vracanje);

    /*
    ToString vraca podatke u formatu:
    oznaka | ime i prezime klijenta | model vozila | preuzimanje -
vracanje
    RC-EK-0042-AB | Amina Buric | VW Golf | 01.06.2026 08:00 -
06.06.2026 08:00
    */
    cout << najam1.ToString() << crt;

    /*
    DodajStavku dodaje dodatnu stavku (naziv, cijena) samo ako je
cijena > 0.
    U zavisnosti od uspjesnosti funkcija vraca true ili false.


    IzracunajCijenu:
    --------------------
    osnovnaCijena = dnevna tarifa * broj dana;
    --------------------
    popust prema broju dana najma:
    >=30 -> 25%
    >=14 -> 15%
    >=7 -> 10%
    --------------------
    doplata za mladog vozaca:
    mladji od 25 godina u godini preuzimanja +20% na osnovnu cijenu
```

```
    (prije popusta) + zbir svih dodatnih stavki.

    npr. Amina (rodj. 2005) u 2026. ima 21 god -> mladi vozac;
    5 dana * 60 KM = 300 KM;
    300 KM + 20% = 360 KM. (mladi vozac);
    nema popusta na broj dana najma;
    + dodatne stavke ukoliko postoje
    */
    cout << "Cijena (mladi vozac, 5 dana): " <<
najam1.IzracunajCijenu()
        << " KM" << crt;
    najam1.DodajStavku("GPS", 10);
    najam1.DodajStavku("Djecije sjediste", 15);
    cout << "Cijena sa stavkama: " << najam1.IzracunajCijenu() << "
KM" << crt;

    Najam najam2(goran, passat, 7,
        DatumVrijeme(1, 7, 2026), DatumVrijeme(11, 7, 2026));
    cout << "Cijena (10 dana, popust 10%): " <<
najam2.IzracunajCijenu()
        << " KM" << crt;

    /*
     SePreklapaSa provjerava da li se dva najma preklapaju. Vraca true
samo ako
     se oba najma odnose na ISTO vozilo (ista registracija) i ako im
se periodi
     preuzimanje i vracanje vremenski preklapaju.

     Periodi se NE preklapaju jedino kada je jedan u cijelosti prije
drugog -
     tj. kada jedan najam pocinje tek nakon sto drugi zavrsi. Kada
     jedan zavrsava tacno u trenutku kada drugi pocinje, racuna se kao
     preklapanje.
     */
    Najam istiGolfPreklapa(goran, golf, 8,
        DatumVrijeme(4, 6, 2026), DatumVrijeme(9, 6, 2026));
    Najam istiGolfKasnije(goran, golf, 9,
        DatumVrijeme(10, 6, 2026), DatumVrijeme(15, 6, 2026));
    if (najam1.SePreklapaSa(istiGolfPreklapa))
        cout << "Preklapanje detektovano" << crt;
    if (!najam1.SePreklapaSa(istiGolfKasnije))
        cout << "Nema preklapanja (razliciti periodi)" << crt;
    if (!najam1.SePreklapaSa(najam2))
        cout << "Nema preklapanja (razlicito vozilo)" << crt;

    Najam kopijaNajma = najam1;
    cout << kopijaNajma << crt;
    Najam dodijeljeniNajam;
    dodijeljeniNajam = najam1;
    cout << dodijeljeniNajam << crt;

    Agencija agencija("Rent Fast");

    /*
    DodajNajam baca izuzetak u slucaju da se pokusa dodati neispravan
period (datum vracanja
```

```
    prije datuma preuzimanja - ukljucujuci i slucaj jednakih termina)
ili postoji
    preklapanje istog vozila u vec zauzetom periodu.
    */
    agencija.DodajNajam(najam1);
    agencija.DodajNajam(najam2);
    try {
        agencija.DodajNajam(istiGolfPreklapa);
    }
    catch (exception& e) {
        cout << "Exception: " << e.what() << crt;
    }
    try {
        Najam losPeriod(amina, passat, 10,
            DatumVrijeme(5, 8, 2026), DatumVrijeme(1, 8, 2026));
        agencija.DodajNajam(losPeriod);
    }
    catch (exception& e) {
        cout << "Exception: " << e.what() << crt;
    }
    agencija.DodajNajam(istiGolfKasnije);
    cout << "Broj najmova: " << agencija.GetNajmovi().size() << crt;

    /*
    JeVoziloSlobodno provjerava da li postoji preklapajuci najma za
dato vozilo u
    trazenom periodu.
    */
    if (!agencija.JeVoziloSlobodno("A12-B-345",
        DatumVrijeme(2, 6, 2026), DatumVrijeme(4, 6, 2026)))
        cout << "Golf zauzet 02-04.06." << crt;
    if (agencija.JeVoziloSlobodno("A12-B-345",
        DatumVrijeme(20, 6, 2026), DatumVrijeme(25, 6, 2026)))
        cout << "Golf slobodan 20-25.06." << crt;

    /*
    PronadjiNajam vraca pokazivac na najam sa datom oznakom ili
nullptr.
    */
    string oznaka1 = najam1.GetOznaka();
    if (agencija.PronadjiNajam(oznaka1) != nullptr)
        cout << "Najam pronadjen: " << oznaka1 << crt;
    if (agencija.PronadjiNajam("RC-XX-9999-XX") == nullptr)
        cout << "Najam nije pronadjen" << crt;

    /*
    EvidentirajVracanje: ako je vraceno nakon planiranog datuma,
dodaje se
    kasnina (dani kasnjenja * 1.5 * dnevna tarifa) na cijenu najma, a
racun se
    salje u zasebnom threadu. Vraca false za nepostojeci najam.

    Primjer sadrzaja racuna (salje se u zasebnom threadu):
    ----------------------------------------
    To: amina@mail.ba
    From: racuni@rentacar.ba
    Subject: Racun za zavrseni najam
```

```
    Postovani Amina Buric,

    Najam RC-EK-0042-AB za vozilo VW Golf je zavrsen.
    Ukupan iznos za naplatu: 385.00 KM

    Hvala na povjerenju.
    ----------------------------------------
    */
    if (agencija.EvidentirajVracanje(oznaka1, DatumVrijeme(6, 6, 2026,
8, 0)))
        cout << "Vracanje na vrijeme evidentirano" << crt;
    if (agencija.EvidentirajVracanje(najam2.GetOznaka(),
        DatumVrijeme(13, 7, 2026)))
        cout << "Vracanje sa kasnjenjem evidentirano" << crt;
    if                  (!agencija.EvidentirajVracanje("RC-XX-9999-XX",
DatumVrijeme()))
        cout << "Vracanje za nepostojeci najam odbijeno" << crt;

    Agencija kopijaAgencije = agencija;
    cout << kopijaAgencije.GetNaziv() << " ima "
        << kopijaAgencije.GetNajmovi().size() << " najmova" << crt;

    cin.get();
    return 0;
}
```